



Hacer consultas desde Python

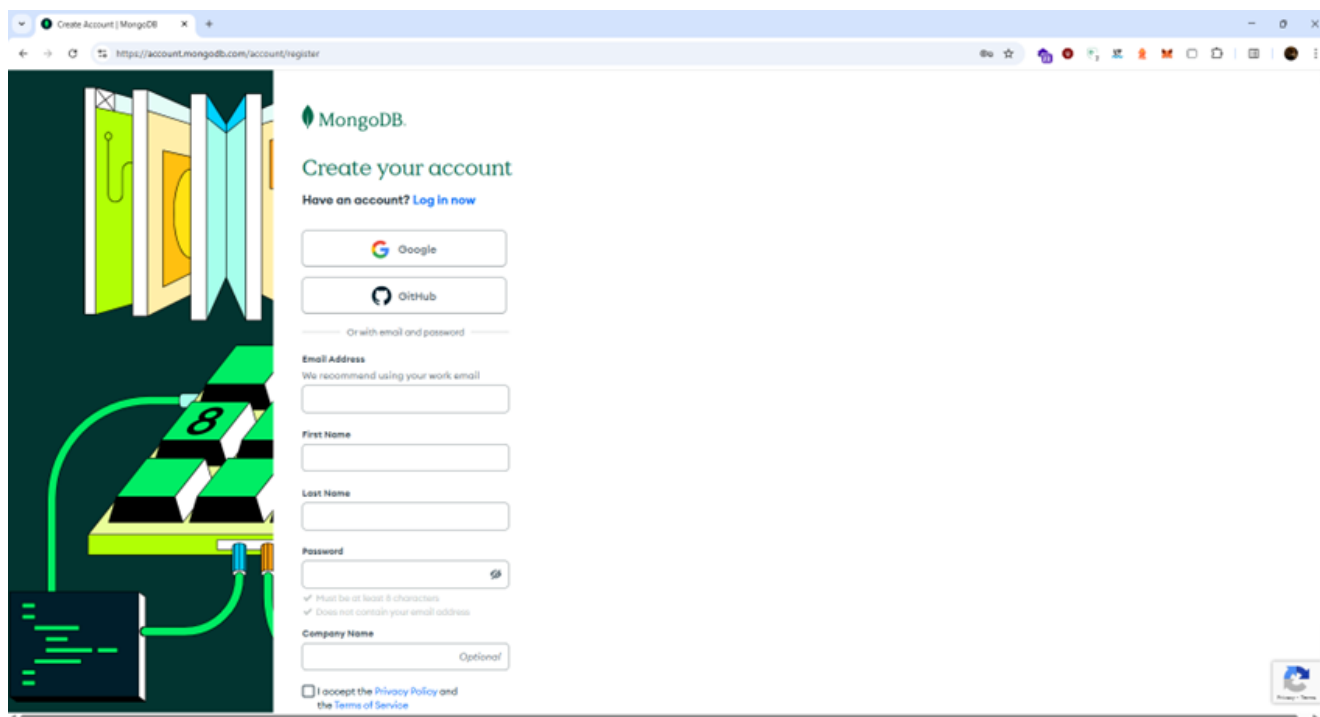
Aprender a conectar una base de datos MongoDB con Python puede facilitar el análisis de información desde entornos de desarrollo accesibles y sin instalaciones locales. En este tema, exploraremos cómo lograr esta conexión a través de una serie de comandos y configuraciones. Al finalizar, serás capaz de importar datos, conectarte a MongoDB y realizar consultas directamente desde Python.

Para conectarnos a la base de datos desde Python, utilizaremos **MongoDB Atlas**, una versión en la nube de la base de datos que ofrece una versión gratuita con las características suficientes para guardar colecciones pequeñas.

Importante:

En esta lección se asume que cuentas con una cuenta de Google o Gmail, así como con experiencia previa en el uso de Google Drive, Google Colaboratory y Python.

El primer paso es crear una cuenta en MongoDB Atlas. Para esto, es necesario registrarse en la página oficial. Encontrarás el enlace para acceder a este sitio en la sección de Material adicional, al terminar esta lectura.

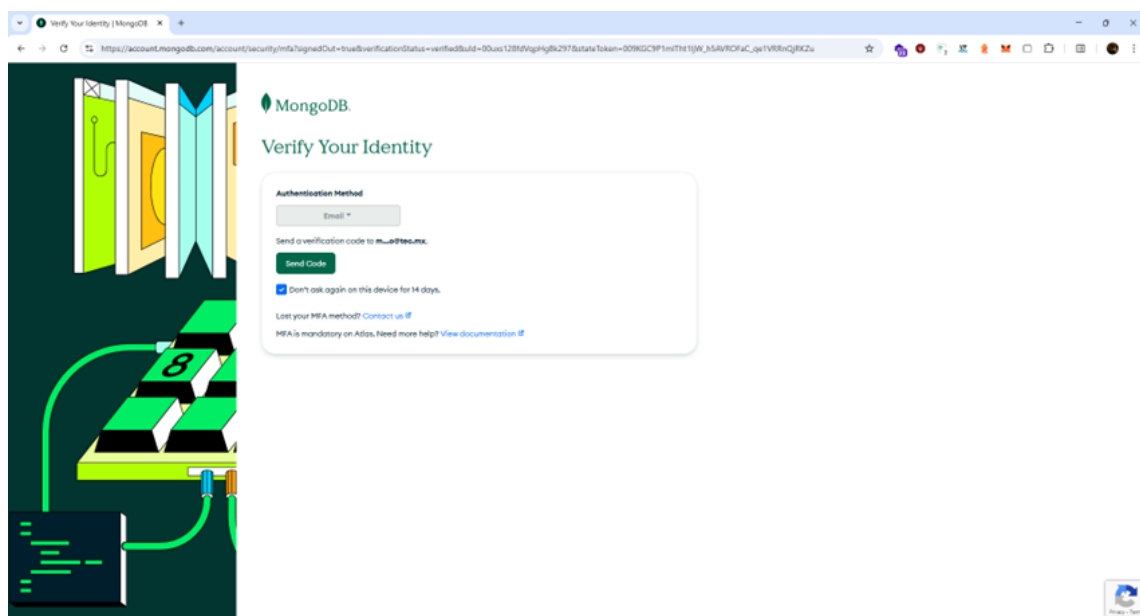


The screenshot shows the MongoDB Atlas 'Create your account' page. On the left is a colorful illustration of a server rack with a green and black checkered path leading to a server unit. The main content area has the MongoDB logo and the title 'Create your account'. Below the title is a link 'Have an account? Log in now'. There are two large buttons for 'Google' and 'OAuth'. Below these is a link 'Or with email and password'. The 'Email Address' section has a note 'We recommend using your work email' and a text input field. Below that are 'First Name' and 'Last Name' text input fields. The 'Password' section has a text input field with a strength indicator and two checkmarks: 'Must be at least 8 characters' and 'Does not contain your email address'. Below the password field is a 'Company Name' text input field with a note 'Optional'. At the bottom, there is a checkbox for 'I accept the Privacy Policy and the Terms of Service'.

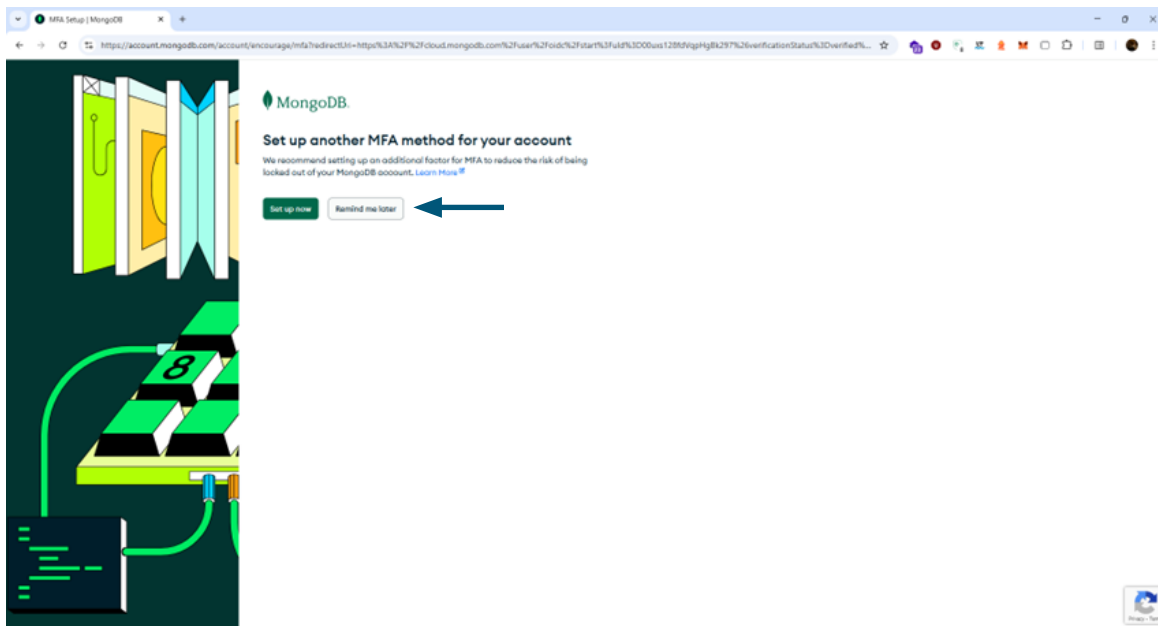
Una vez creada la cuenta, es necesario verificar el correo electrónico y después iniciar sesión.



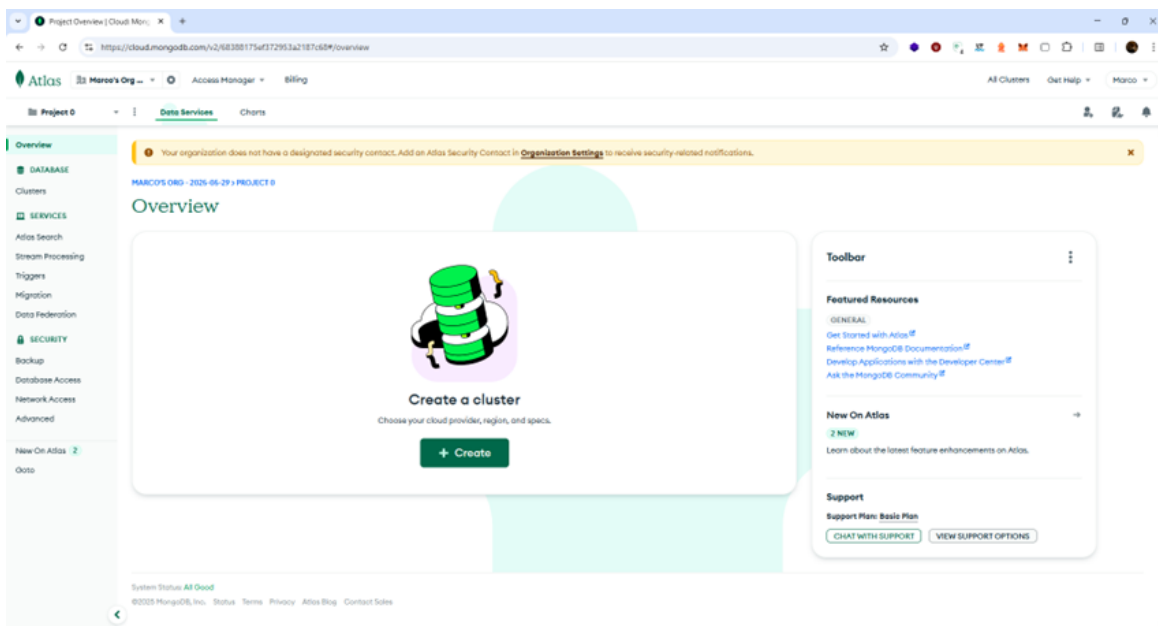
MongoDB requerirá un segundo factor de seguridad, y enviará un código al correo electrónico registrado.



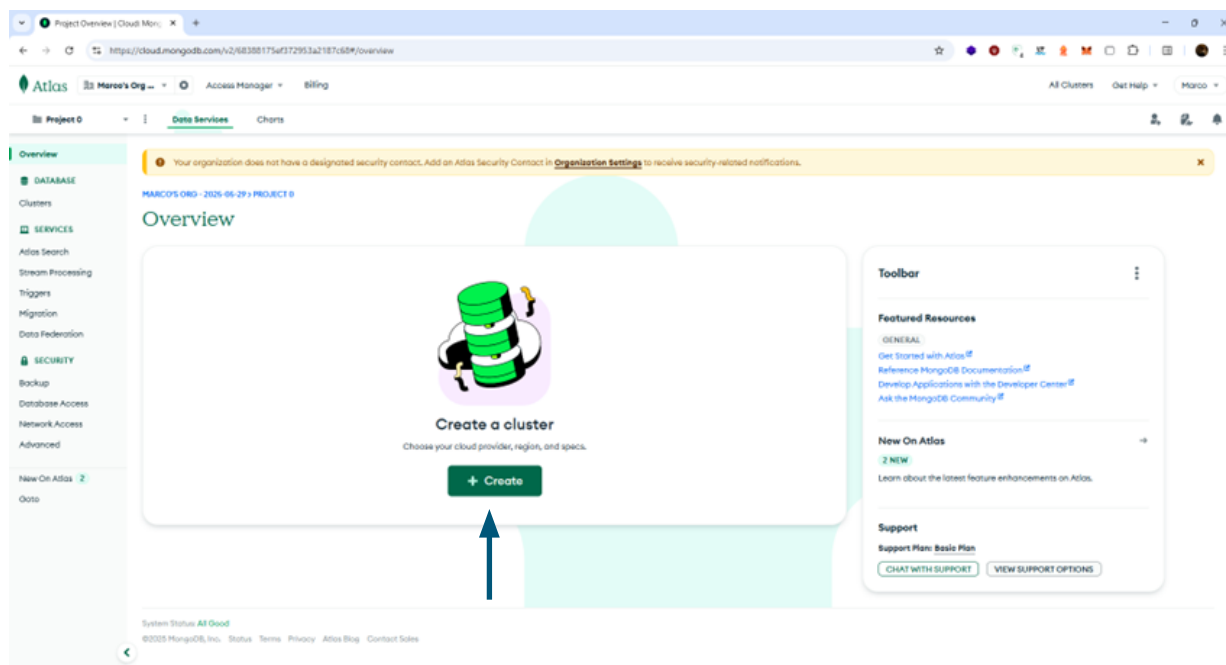
En la siguiente pantalla, sugerirá configurar otro método adicional de autenticación. Este paso puede realizarse después.



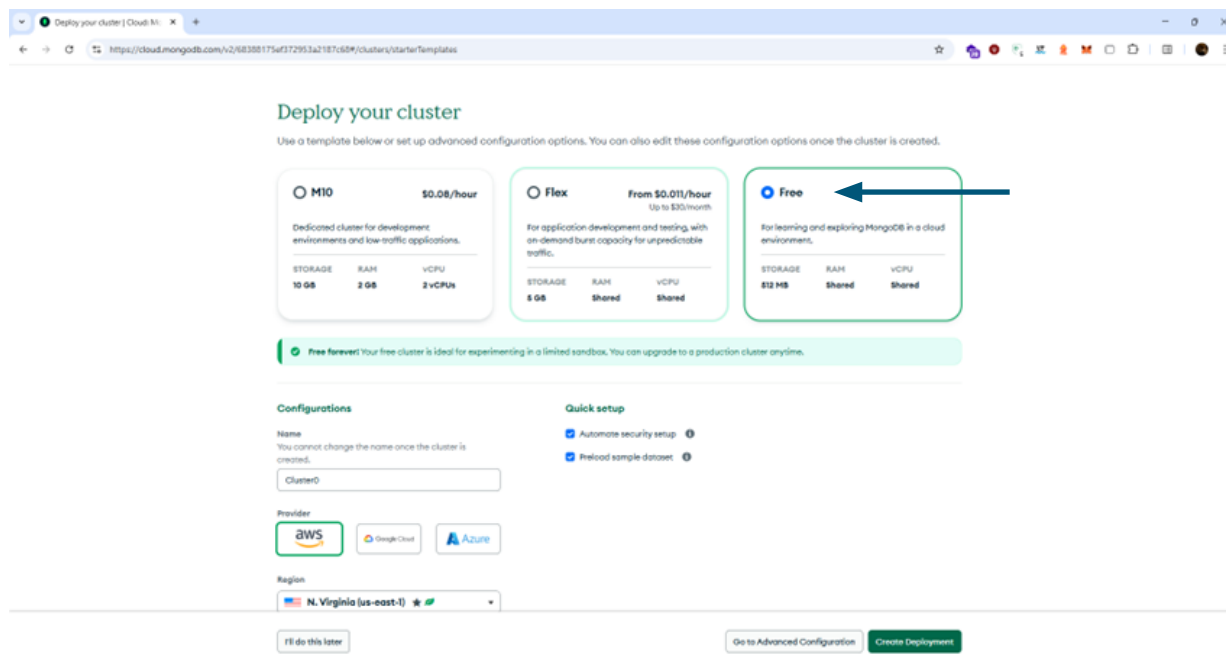
La consola principal de Atlas se muestra a continuación:



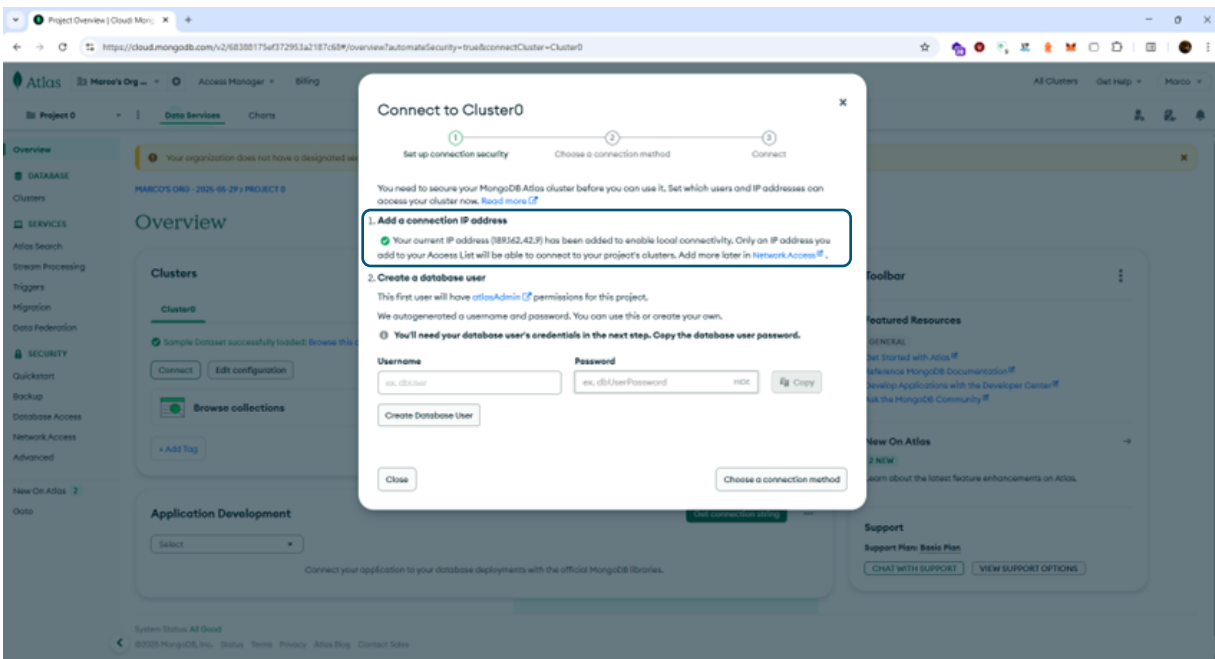
Para poder crear una base de datos, primero se debe crear un **clúster**, que es un conjunto de servidores donde estará alojada la base de datos:



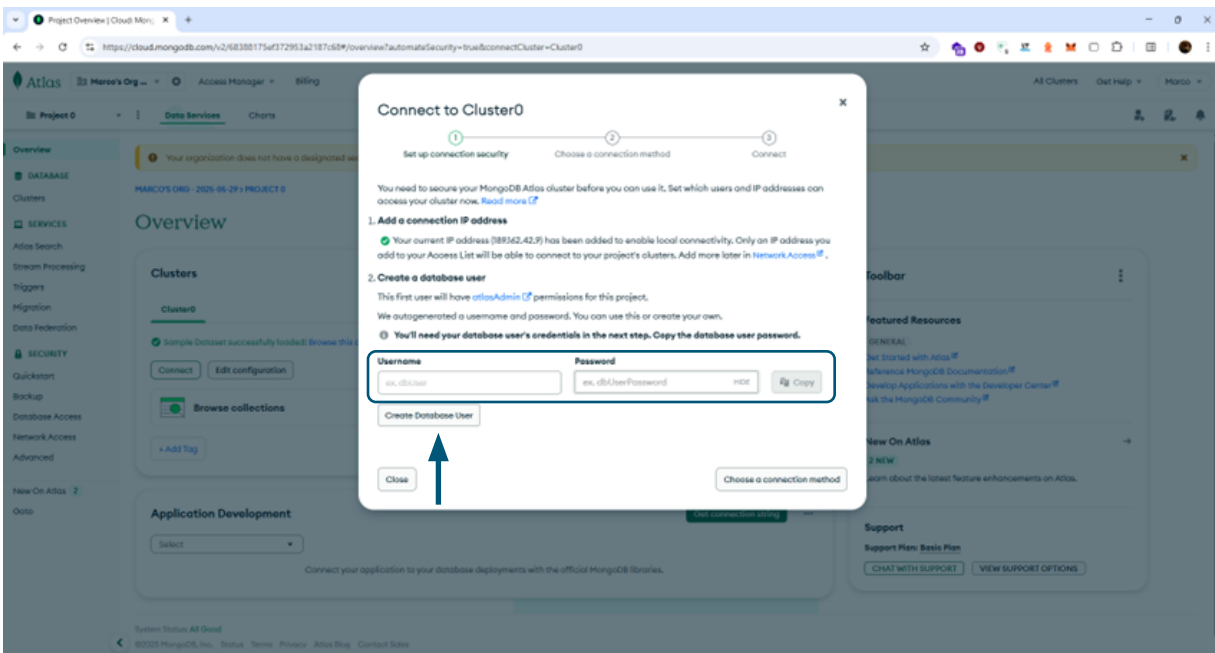
Selecciona el clúster “Free”:



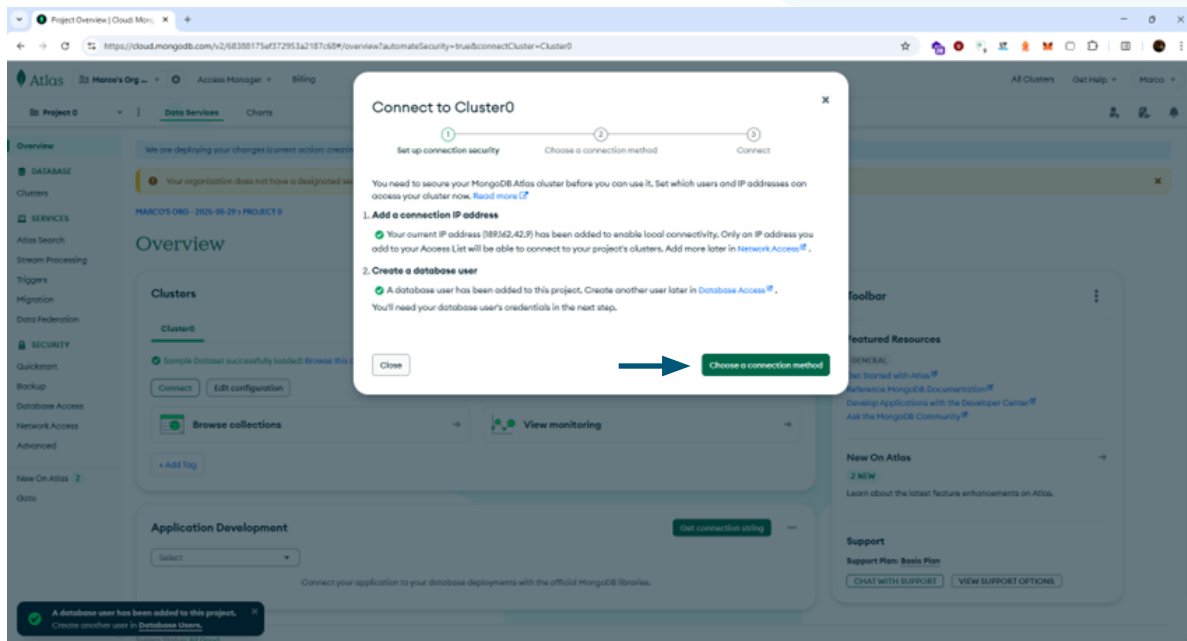
A continuación, se mostrará una pantalla con indicaciones para conectarse al clúster, en la que se señala que se le dio acceso a la dirección actual de internet:



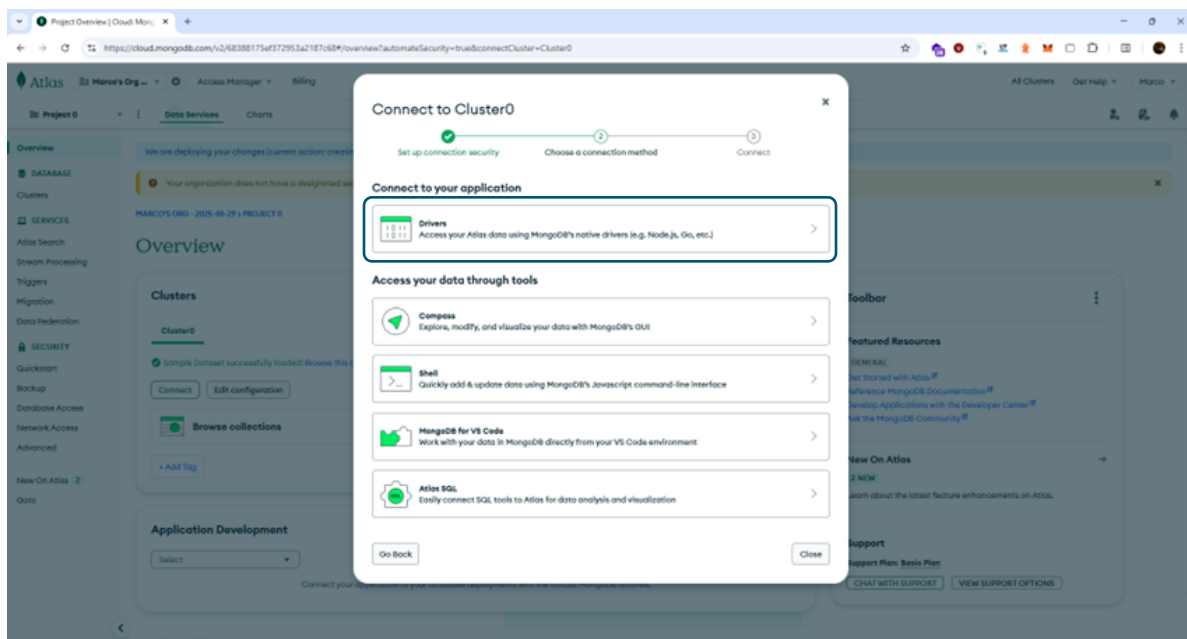
En esta misma pantalla, debes crear un usuario para acceder de forma remota a la base de datos. Recuerda generar una contraseña segura.



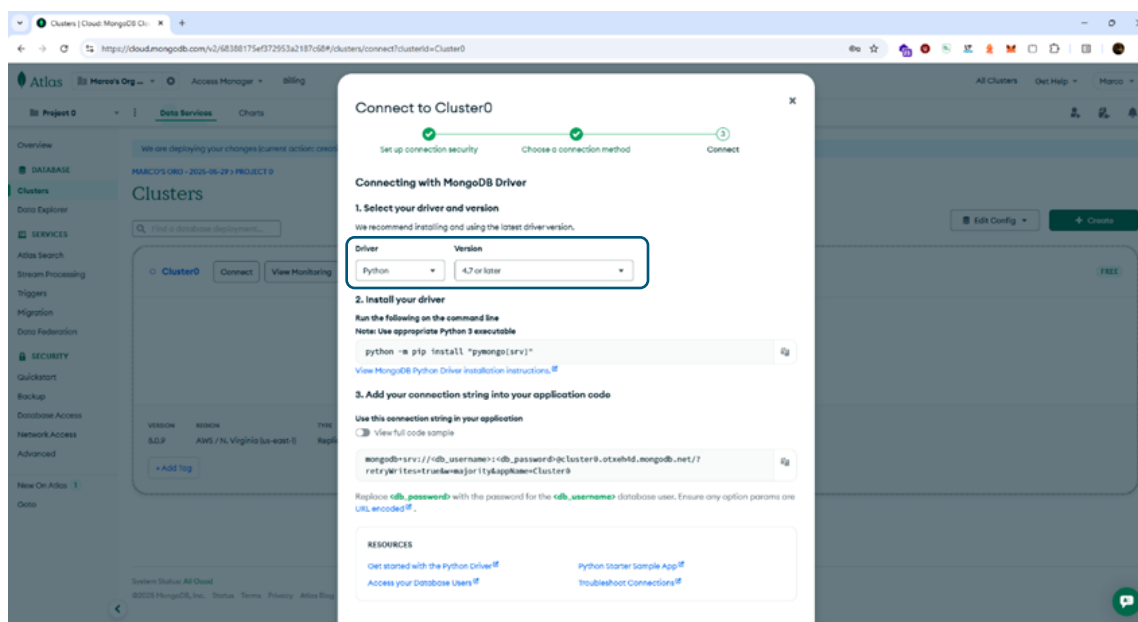
Después de crear el usuario, selecciona el botón **“Choose a connection method”**.



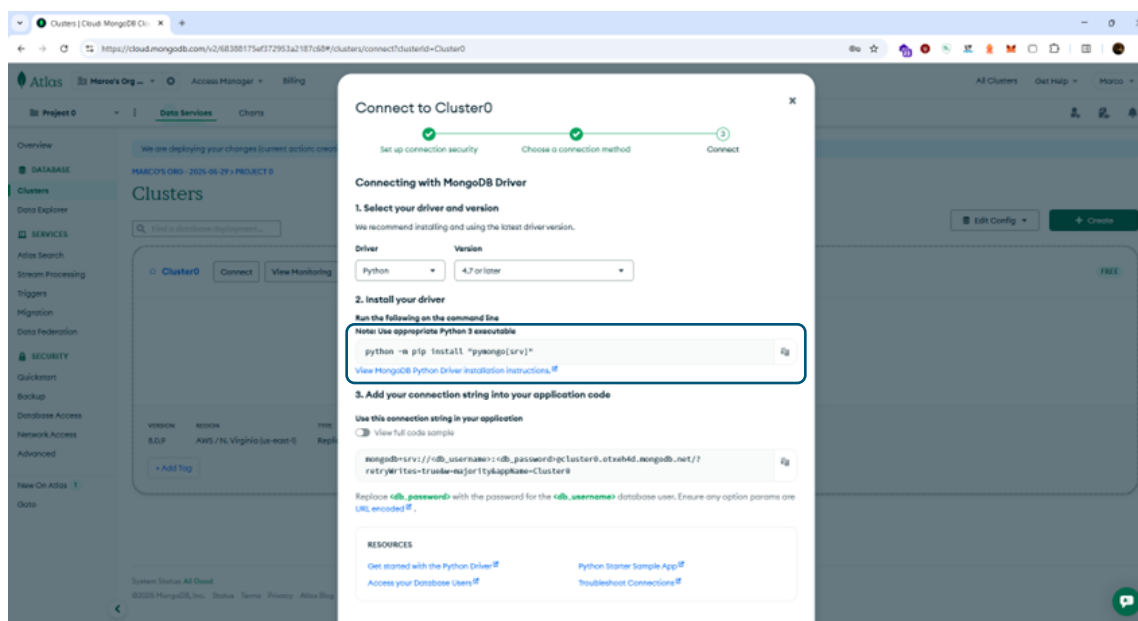
Después, selecciona la opción **“Drivers”**.



Esta pantalla muestra las instrucciones para realizar la conexión con múltiples lenguajes de programación. Selecciona Python y la versión más reciente del *driver*.



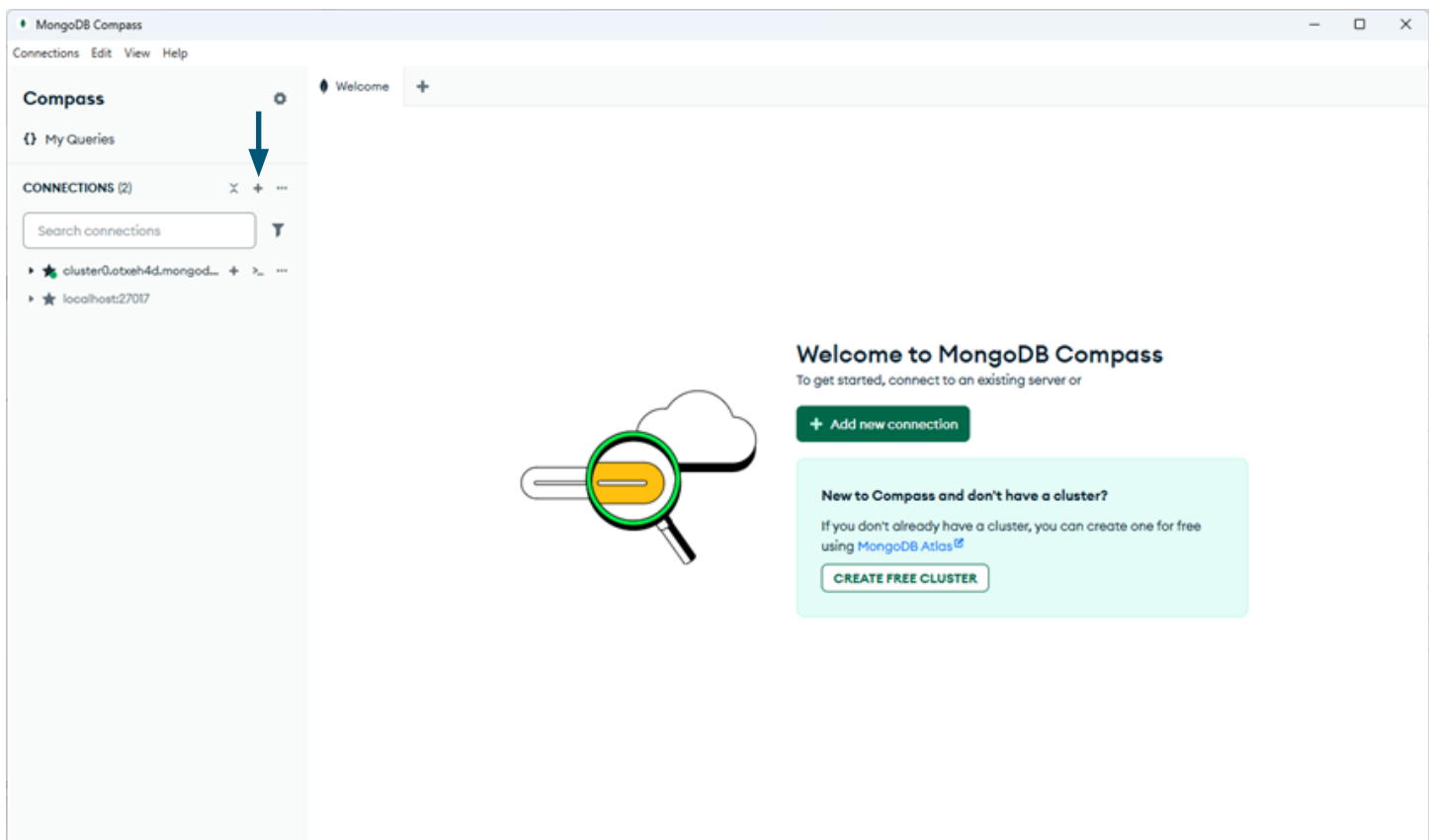
Atlas mostrará cómo instalar el *driver* en Python. Toma nota de este comando, ya que se utilizará en Google Colab.



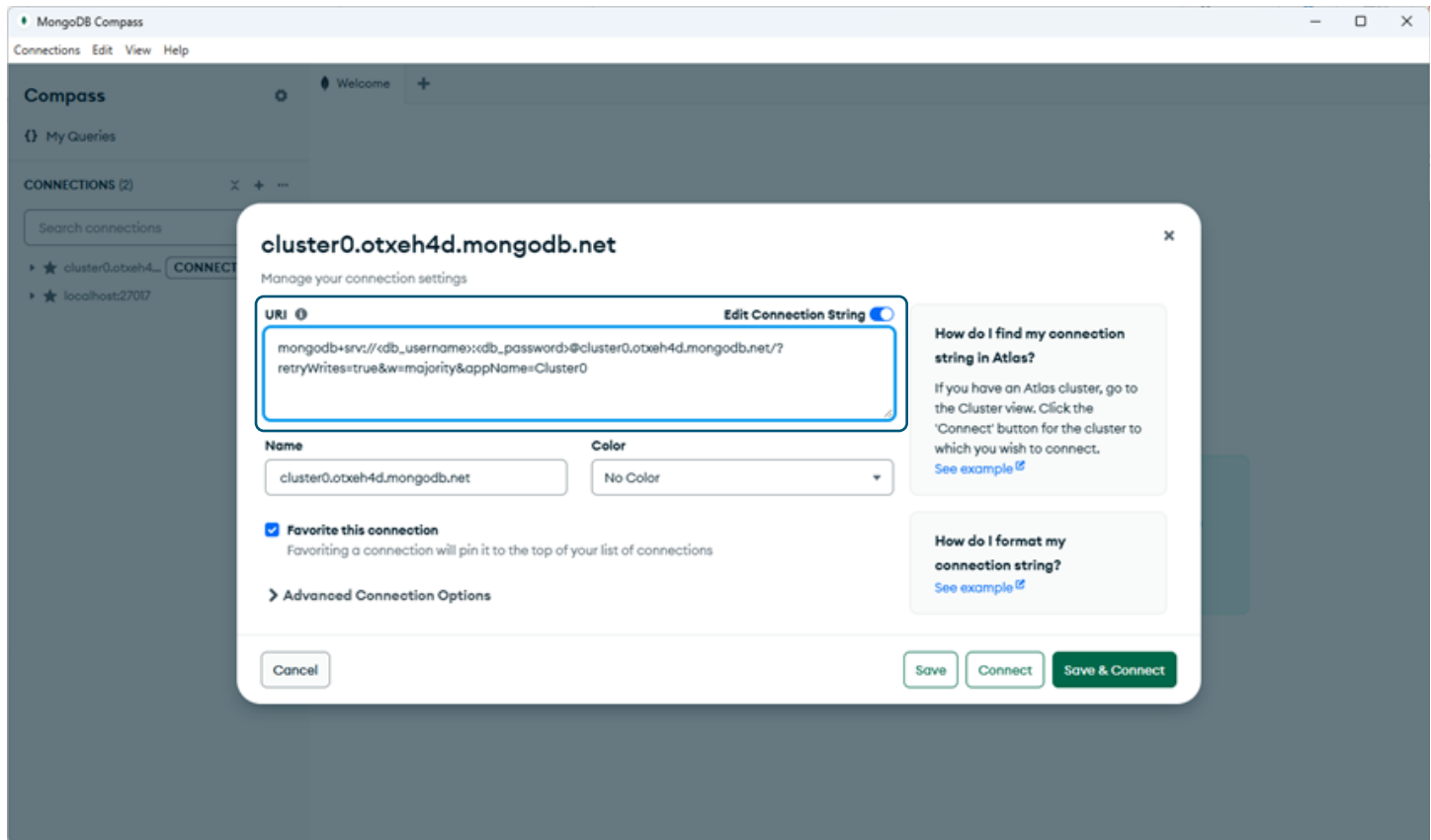
También mostrará la cadena de conexión que utilizaremos. Recuerda reemplazar `<db_username>` y `<db_password>` por el usuario y la contraseña que creaste en el paso anterior, respectivamente.

Ahora, vamos a importar los datos. Para esto, utilizaremos Mongo Compass de la misma manera que en las lecciones anteriores. Sin embargo, la cadena de conexión será la que Atlas nos desplegó en el paso anterior.

Primero, crea una nueva conexión:



Después, copia la cadena de conexión. Recuerda poner el usuario y la contraseña para tu servidor.



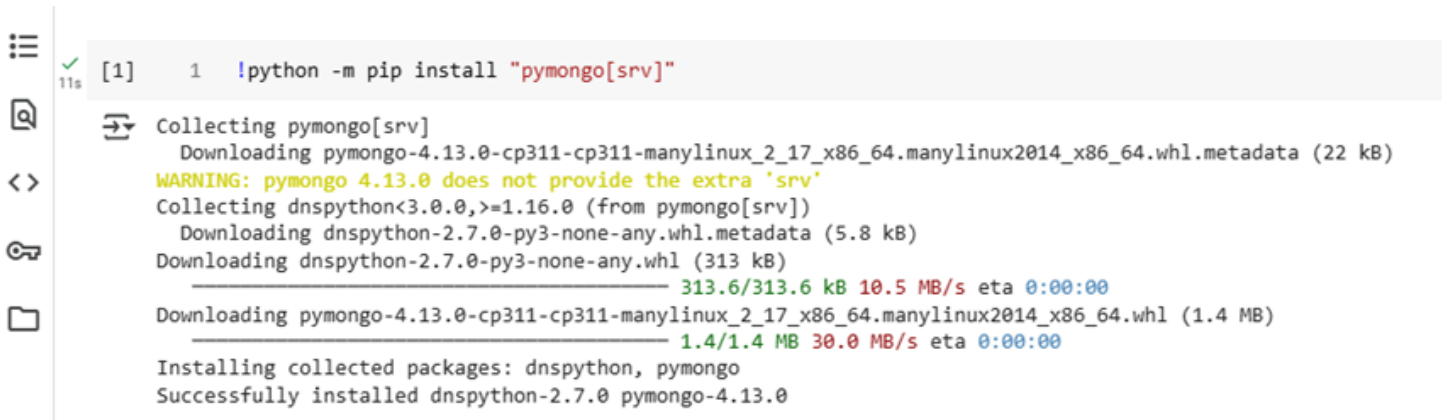
Al igual que en la lección anterior, el conjunto de datos que utilizaremos es el *Nobel Prize Dataset* de Kaggle. Encontrarás la liga para acceder a este sitio en la sección de Material adicional, al terminar esta lectura. Es necesario descargar los archivos `json_award.json` y `json_laureates.json`.

Después, crea una base de datos llamada `nobel` y dos colecciones: `awards` y `laureates`, donde importarás los archivos descargados.

Para acceder a MongoDB desde Python en Google Colab, utilizaremos la biblioteca **PyMongo**, que nos permite conectarnos directamente a la base de datos.

Para instalar la biblioteca, en una celda ejecuta el siguiente comando:

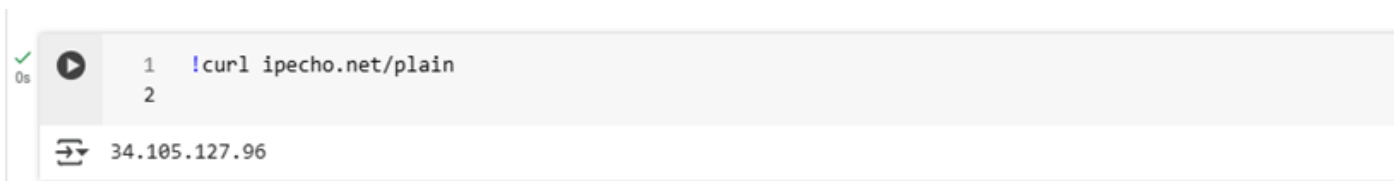
```
!python -m pip install "pymongo[srv]"
```



```
[1] 1 !python -m pip install "pymongo[srv]"
Collecting pymongo[srv]
  Downloading pymongo-4.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (22 kB)
WARNING: pymongo 4.13.0 does not provide the extra 'srv'
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo[srv])
  Downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB)
  Downloading dnspython-2.7.0-py3-none-any.whl (313 kB)
    313.6/313.6 kB 10.5 MB/s eta 0:00:00
  Downloading pymongo-4.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.4 MB)
    1.4/1.4 MB 30.0 MB/s eta 0:00:00
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.7.0 pymongo-4.13.0
```

También es necesario otorgar permisos de acceso a la dirección IP del *notebook* de Google Colab en el clúster de MongoDB. Para obtener esta dirección, ejecuta el siguiente comando:

```
!curl ipecho.net/plain
```



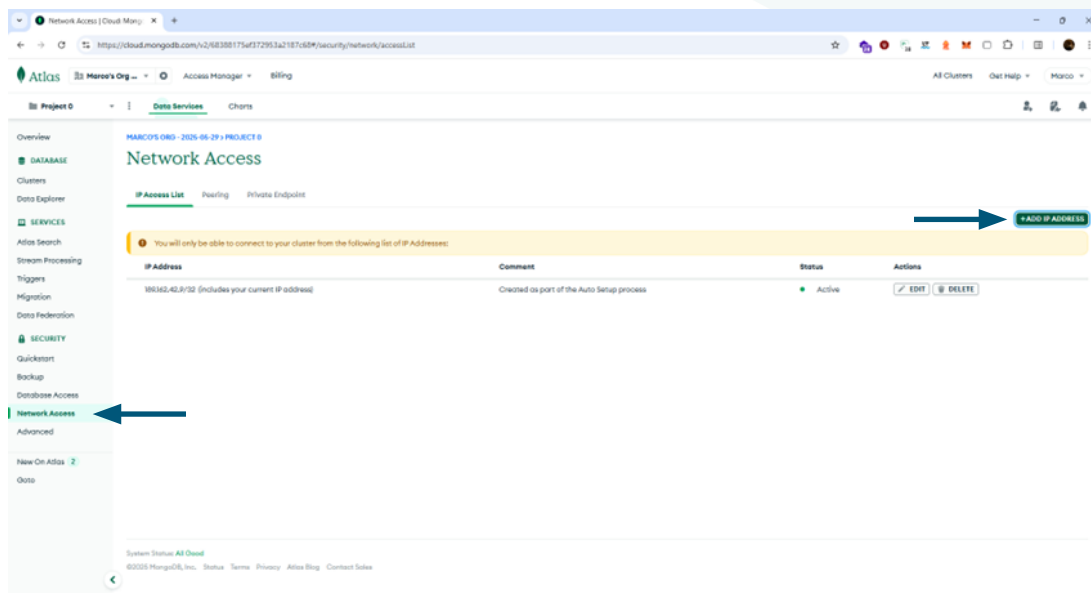
```
1 !curl ipecho.net/plain
2
34.105.127.96
```

El número que se muestra es la dirección IP de tu *notebook*.

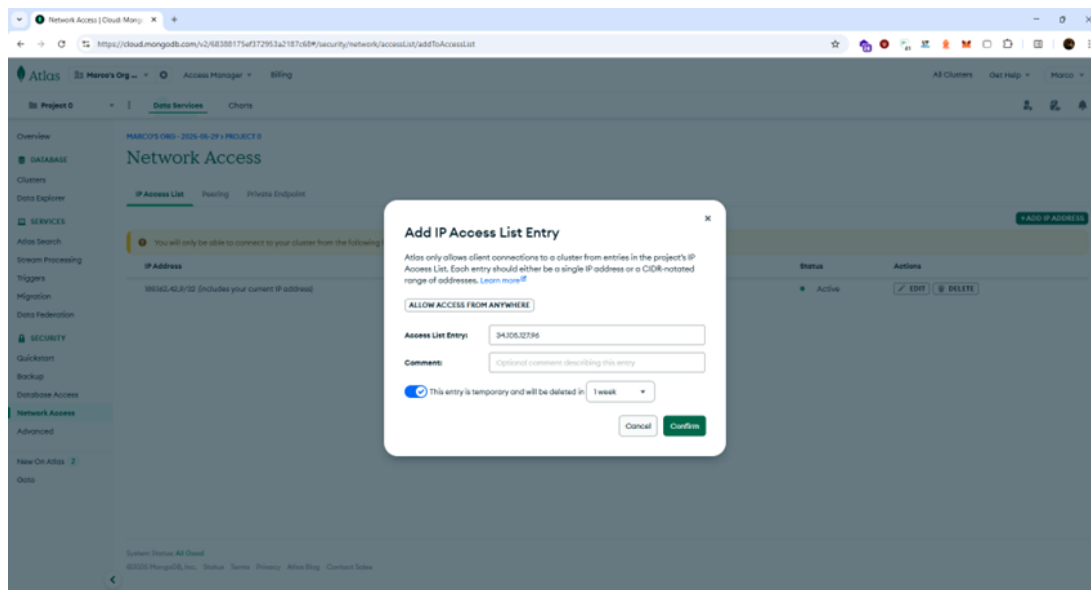
Importante:

Esta dirección IP puede cambiar cada vez que inicias sesión en Google Colab, por lo que debes verificarla y registrarla en cada ocasión.

Para registrar esta dirección, dirígete al panel de Mongo Atlas y selecciona la opción **“Network Access”**. Posteriormente, da clic en **“Add IP Address”**:

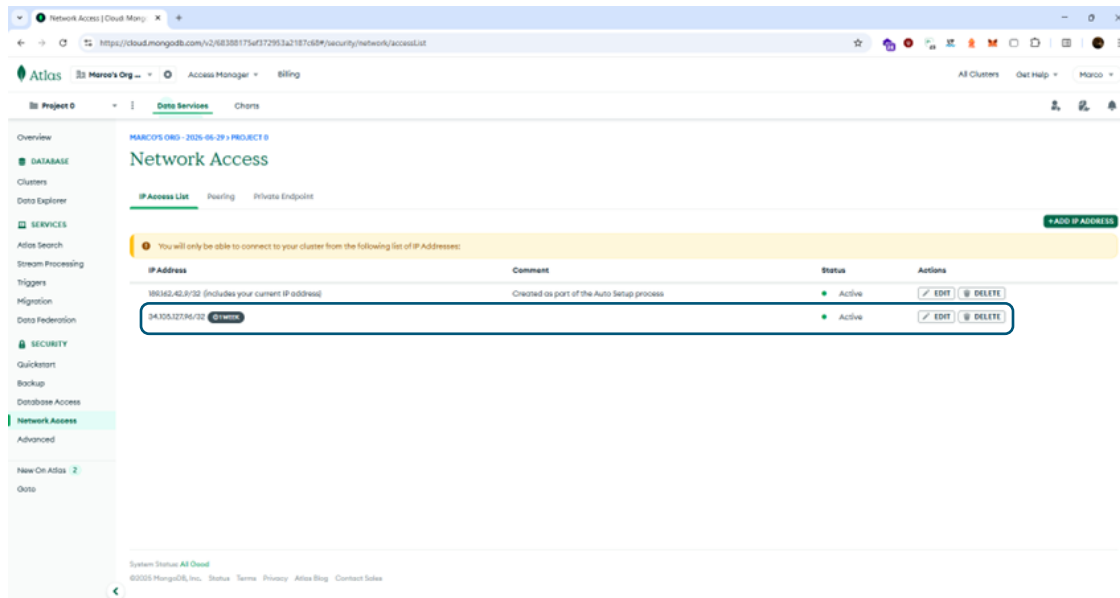


En la siguiente pantalla, introduce la IP de tu *notebook*:



Es recomendable dar acceso temporal para que se borre de manera automática la IP y el servidor no se llene de accesos externos.

Después de confirmar, la consola mostrará la nueva IP en el listado:



Volviendo a Google Colab, para establecer la conexión con el servidor, se debe importar el módulo `pymongo` y crear un objeto `MongoClient` (`servidor`, `puerto`), donde **servidor** es la dirección o nombre del servidor como cadena de caracteres, y **puerto** es el número de puerto donde se ejecuta la base de datos:

```
from pymongo import MongoClient
```

```
client = MongoClient(servidor, 27017)
```

Sin embargo, si el servidor es local (`localhost`) y se está utilizando el puerto estándar (`27017`), ambos parámetros pueden omitirse:

```
client = MongoClient()
```

```
[6] 1 from pymongo import MongoClient
     2 client = MongoClient()
```

Para hacer la conexión con el clúster de Atlas, es necesario utilizar la cadena de conexión desplegada en los pasos anteriores. Recuerda poner el usuario y la contraseña para tu servidor:

```
server = mongodb+srv://<db_username>:  
<db_password>@cluster0.otxeh4d.mongodb.net/?  
retryWrites=true&w=majority&appName=Cluster0"
```

```
from pymongo import MongoClient
```

```
client = MongoClient(server, 27017)
```

```
1 server = "mongodb+srv://<db_username>:<db_password>@cluster0.otxeh4d.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0"  
2 from pymongo import MongoClient  
3  
4 client = MongoClient(server, 27017)  
5  
6
```

Para referenciar una base de datos específica, existen dos opciones:

1. Si el nombre de la base de datos **no tiene espacios ni caracteres especiales**, puede referenciarse como un atributo del objeto cliente. Por ejemplo, para referenciar la base de datos `nobel`, usamos el siguiente código:

```
nobel = client.nobel
```

2. Si el nombre de la base de datos **tiene espacios o caracteres especiales**, debe utilizarse la sintaxis de diccionario:

```
nobel = client['nobel']
```

```
✓ [7] 1 nobel = client.nobel
```

La forma de referenciar una colección es la misma que para una base de datos. Por lo tanto, cualquiera de las siguientes opciones es válida para asignar la colección `laureates` a una variable:

```
laureates = nobel.laureates
```

```
laureates = nobel['laureates']
```

```
✓ [8] 1 laureates = nobel.laureates
```

Para realizar consultas, se utiliza el método `find` de la colección, y se le indica la consulta utilizando un diccionario de Python. La biblioteca **PyMongo** se encarga de transformar ese diccionario en un documento de MongoDB automáticamente. De la misma manera, PyMongo transformará documentos de MongoDB en diccionarios de Python. Aunque la sintaxis es similar, puede haber diferencias sutiles que deben tenerse en cuenta.

Por ejemplo, para obtener a quienes han ganado un premio Nobel y nacieron en Estados Unidos, en MongoDB utilizamos:

```
{"birth.place.country.en": "USA"}
```

Desde Python, la consulta sería de la siguiente forma:

```
usa_laureates=list(laureates.find({"birth.place.country.en":"USA"}))
```

O bien, se puede asignar la consulta a una variable y luego referenciarla:

```
query={"birth.place.country.en":"USA"}
```

```
usa_laureates=list(laureates.find(query))
```

En ambos casos, el resultado será una **lista** de Python que contiene los documentos obtenidos, convertidos en diccionarios.

```
[10] 1 query={"birth.place.country.en":"USA"}
      2 usa_laureates=list(laureates.find(query))
      3
      4 usa_laureates

[{'_id': ObjectId('624933d3950c854270274b31'),
  'birth': {'date': '1943-00-00',
            'place': {'city': {'en': 'Montclair, NJ',
                               'no': 'Montclair, NJ',
                               'se': 'Montclair, NJ'},
                      'cityNow': {'en': 'Montclair, NJ',
                                   'no': 'Montclair, NJ',
                                   'se': 'Montclair, NJ'},
                      'continent': {'en': 'North America'},
                      'country': {'en': 'USA', 'no': 'USA', 'se': 'USA'},
                      'countryNow': {'en': 'USA', 'no': 'USA', 'se': 'USA'},
                      'locationString': {'en': 'Montclair, NJ, USA',
                                           'no': 'Montclair, NJ, USA',
                                           'se': 'Montclair, NJ, USA'}}}},
  'familyName': {'en': 'Spence', 'se': 'Spence'},
  'fullName': {'en': 'A. Michael Spence', 'se': 'A. Michael Spence'},
  'gender': 'male',
  'givenName': {'en': 'A. Michael', 'se': 'A. Michael'},
  'id': '745',
  'knownName': {'en': 'A. Michael Spence', 'se': 'A. Michael Spence'},
  'links': {'action': 'Get',
            'href': 'http://masterdataapi.nobelprize.org/2/laureate/745',
            'rel': 'laureate',
            'types': 'application/json'},
  'nobelPrizes': {'affiliations': {'city': {'en': 'Stanford, CA'}}
```

Sobre la misma consulta, se puede hacer una proyección para obtener solamente los campos que se necesitan:

```
query={"birth.place.country.en":"USA"}
field_projection={"fullName.en":1,"birth.place.country.
en":1,"_id":0 }
usa_laureates=list(laureates.find(query,field_projection))
```

```

✓ [11] 1 query={"birth.place.country.en":"USA"}
      2 field_projection={"fullName.en":1,"birth.place.country.en":1,"_id":0 }
      3 usa_laureates=list(laureates.find(query,field_projection))
      4
      5 usa_laureates

```

```

[{'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'A. Michael Spence'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Adam G. Riess'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Albert Arnold (Al) Gore Jr.'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alan J. Heeger'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alfred D. Hershey'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alfred G. Gilman'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alvin E. Roth'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Andrew Z. Fire'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Arthur Ashkin'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Arthur Holly Compton'}}]

```

✓ 0 s se ejecutó 23:49

El resultado del método `find()` es un objeto iterable llamado **Cursor**, que se puede recorrer con un ciclo `for`, o en este caso, convertir directamente en una lista para manipularla como cualquier otra lista de Python. Por ejemplo, para recuperar los primeros cinco documentos, utilizamos:

```
first_five=usa_laureates[:5]
```

```

✓ [12] 1 first_five=usa_laureates[:5]
      2
      3 first_five

```

```

[{'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'A. Michael Spence'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Adam G. Riess'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Albert Arnold (Al) Gore Jr.'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alan J. Heeger'}},
 {'birth': {'place': {'country': {'en': 'USA'}}},
  'fullName': {'en': 'Alfred D. Hershey'}}]

```

Cualquier consulta válida de MongoDB puede ser ejecutada y recuperarse desde Python, y subsecuentemente se puede recorrer o convertir en una lista.

Otro comando comúnmente utilizado es `count_documents`, que devuelve el número de documentos que coinciden con una consulta. Por ejemplo, para contar el total de documentos en la colección, se puede utilizar la siguiente consulta:

```
laureates.count_documents({})
```

```
✓ [13] 1 laureates.count_documents({})  
943
```

Antes de continuar...

Al ser un lenguaje de programación de propósito general, Python provee diversas herramientas para manipular información. Con esto en mente, reflexiona:

- ¿Cómo crees que se pueda integrar Python en un ambiente de análisis en una empresa?
- ¿Qué ventajas y desventajas puede tener Python para acceder y manipular la información en MongoDB en comparación con Compass?

Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.

Material adicional

Devakumar K. P. (2020). *Nobel Prize Dataset* [Conjunto de datos]. Kaggle. Recuperado el 11 de junio de 2025 de: <https://www.kaggle.com/imdevskp/nobel-prize>

MongoDB. (s.f.). *Create your account* [Crea tu cuenta]. MongoDB. Recuperado el 16 de junio de 2025 de: <https://account.mongodb.com/account/register>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 16 de junio de 2025 y su disponibilidad depende del autor.

Glosario

Cursor: Objeto iterable devuelto por el método `find()` en PyMongo. Representa los resultados de una consulta y puede ser recorrido con ciclos o convertido en una lista.

Google Colaboratory (Colab): Entorno de desarrollo en la nube proporcionado por Google, que permite ejecutar código en Python sin necesidad de instalar *software* localmente. Ideal para trabajar con *notebooks* y realizar pruebas con bibliotecas como PyMongo y servidores como MongoDB.

Puerto (Port): Número lógico que representa un punto de conexión para servicios de red. MongoDB por defecto utiliza el puerto 27017.

PyMongo: Biblioteca de Python que facilita la conexión y manipulación de bases de datos MongoDB desde aplicaciones Python. Permite realizar consultas, inserciones y operaciones sobre colecciones.

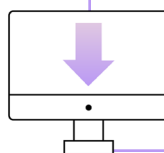
Bibliografía

Bionichive. (s.f.). *BSON Specification* [Especificación BSON]. Bionichive. Recuperado el 11 de junio de 2025 de: <https://www.bionichive.com/>

MongoDB Inc. (s.f.). *Documentación de MongoDB*. MongoDB. Recuperado el 11 de junio de 2025 de: <https://docs.mongodb.com/>

MongoDB Inc. (s.f.). *PyMongo 4.13.0 Documentation*. MongoDB. Recuperado el 11 de junio de 2025 de: <https://pymongo.readthedocs.io/en/stable/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 11 de junio de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.